

OC Engine

- [1. Kompakte System-Referenz für KI-Modelle](#)
- [2. Operationen](#)
- [3. EXEC – Kommando-Ausführung im Pod](#)
 - [3.1 Parameter](#)
- [4. Pod-Ermittlung](#)
- [5. Namespace](#)
- [6. Lazy Login](#)
- [7. Automatische Exit-Code-Prüfung](#)
 - [7.1 Erwarteter Non-Zero Exit-Code](#)
 - [7.2 Automatische Prüfung deaktivieren](#)
- [8. Timeout](#)
- [9. outputFile](#)
 - [Relative Pfade](#)
- [10. PUT – Datei in einen Pod kopieren](#)
 - [10.1 PUT Parameter](#)
- [11. GET – Datei aus einem Pod kopieren](#)
 - [11.1 GET Parameter](#)
- [12. Exit-Code-Prüfung bei PUT und GET](#)
- [13. Windows-Pfade bei PUT und GET](#)
- [14. ASSERT – Validierung](#)
 - [EXITCODE](#)
 - [CONTAINS](#)
 - [NOT CONTAINS](#)
 - [COUNT](#)
- [15. BUFFER – Datenextraktion](#)
 - [15.1 BUFFER Actions](#)
 - [FILTER](#)
 - [LINE](#)
 - [LAST_LINE](#)
 - [COUNT](#)
- [16. Vollständiges Beispiel](#)
- [17. Troubleshooting & Best Practices](#)
 - [Pod nicht gefunden](#)
 - [Unerwarteter Exit-Code](#)
 - [Timeout](#)
 - [PUT/GET schlägt fehl](#)
 - [outputFile und GET nicht verwechseln](#)
- [18. Kurzreferenz](#)

Die **OC Engine** ermöglicht die direkte Interaktion mit OpenShift-Clustern.

Sie unterstützt:

- die Ausführung von Shell-Kommandos innerhalb von Pods,
- die automatische Ermittlung laufender Pods,
- den Datei-Transfer zwischen AGATE und Pods,

- automatische Exit-Code-Prüfung,
- Timeouts,
- die lokale Speicherung von Kommando-Outputs,
- die Extraktion von Daten über `BUFFER` ,
- die Validierung von Ergebnissen über `ASSERT` .

1. Kompakte System-Referenz für KI-Modelle

```

1 # AGATE OC-Engine Syntax Referenz
2 # Nutze dies für Testfall-Generierung und Analyse
3
4 # EXEC
5 - type: OC
6   op: EXEC
7   pod: "..."
8   namespace: "..." # Optional, Default:
{E[env.openShift.namespace]}
9   command: "..."
10  response: "..." # Optional
11  expectedExitCode: 0 # Optional, Default: 0
12  checkExitCode: true # Optional, Default: true
13  timeout: 30 # Optional, Sekunden, Default: 30
14  outputFile: "..." # Optional, lokale Datei auf AGATE-Seite
15  condition: "..." # Optional
16
17 # PUT
18 - type: OC
19   op: PUT
20   pod: "..."
21   namespace: "..."
22   from: "..." # lokal
23   to: "..." # remote
24   response: "..."
25   expectedExitCode: 0 # Optional, Default: 0
26   checkExitCode: true # Optional, Default: true
27   timeout: 30 # Optional, Default: 30
28   condition: "..."
29
30 # GET
31 - type: OC
32   op: GET
33   pod: "..."
34   namespace: "..."
35   from: "..." # remote
36   to: "..." # lokal
37   response: "..."
38   expectedExitCode: 0
39   checkExitCode: true
40   timeout: 30
41   condition: "..."
42
43 # BUFFER
44 - type: OC
45   op: BUFFER
46   response: "..."
47   action: <TEXT|FILTER|LINE|LAST_LINE|COUNT>
48   value: "..."
49   name: "..."
50   condition: "..."
51

```

```
52 # ASSERT
53 - type: OC
54   op: ASSERT
55   response: "..."
56   action: <EXITCODE|CONTAINS|NOT_CONTAINS|EQUALS|NOT_EQUALS|COUNT>
57   value: "..."
58   expected: "..."
59   condition: "..."
60
```

2. Operationen

Operation	Beschreibung
EXEC	Führt ein Shell-Kommando innerhalb eines OpenShift-Pods aus.
PUT	Kopiert eine lokale Datei in einen Pod.
GET	Kopiert eine Datei aus einem Pod auf das lokale AGATE-System.
BUFFER	Extrahiert Informationen aus dem Output eines vorherigen Schritts.
ASSERT	Validiert Exit-Code oder Output eines vorherigen Schritts.

3. EXEC – Kommando-Ausführung im Pod

EXEC führt einen Befehl innerhalb eines laufenden OpenShift-Pods aus.

Beispiel:

```
1 - type: OC
2   op: EXEC
3   pod: "my-service"
4   command: "java -version"
5   response: java_out
6
```

Die Engine erfasst:

- Exit-Code
- stdout/stderr
- Timeout-Status

- Ausführungsdauer

Das Ergebnis kann unter `response` gespeichert und anschließend mit `BUFFER` oder `ASSERT` weiterverarbeitet werden.

3.1 Parameter

Parameter	Beschreibung	Erforderlich	Default
<code>pod</code>	Basis-Name bzw. Präfix des Ziel-Pods	Ja	–
<code>command</code>	Im Pod auszuführender Shell-Befehl	Ja	–
<code>namespace</code>	OpenShift- Namespace	Nein	<code>{E[env.openshift.namespace]}</code>
<code>response</code>	Name zum Speichern des Ergebnisses	Nein	–
<code>expectedExitCode</code>	Erwarteter Exit- Code	Nein	<code>0</code>
<code>checkExitCode</code>	Automatische Exit-Code- Prüfung	Nein	<code>true</code>
<code>timeout</code>	Maximale Ausführungszeit in Sekunden	Nein	<code>30</code>
<code>outputFile</code>	Lokale Datei für den Kommando- Output	Nein	–
<code>condition</code>	Bedingung für die Ausführung	Nein	–

4. Pod-Ermittlung

Der in `pod` angegebene Wert muss nicht der vollständige technische Pod-Name sein.

Beispiel:

```
1 pod: "ecsraum2bebatch"
2
```

Ein tatsächlicher Pod kann beispielsweise heißen:

```
1 ecsraum2bebatch-779444f45b-fxht9
2
```

Die OC Engine ermittelt zur Laufzeit automatisch einen passenden laufenden Pod.

Dadurch müssen Testfälle keine dynamisch generierten OpenShift-Pod-Namen enthalten.

5. Namespace

Der Namespace kann explizit angegeben werden:

```
1 - type: OC
2   op: EXEC
3   namespace: "kvw-int-app"
4   pod: "kvw-pst-pst"
5   command: "pwd"
6
```

Wird kein `namespace` angegeben, verwendet die Engine die konfigurierte OpenShift-Umgebung, beispielsweise:

```
1 {E[env.openShift.namespace]}
2
```

Dadurch können dieselben Tests gegen unterschiedliche Stages ausgeführt werden, ohne den Namespace im Testfall hart zu kodieren.

6. Lazy Login

Die OC Engine erkennt eine fehlende bzw. abgelaufene OpenShift-Session und kann bei Bedarf einen Login mit den für die Umgebung konfigurierten Credentials durchführen.

Beispielkonfiguration:

```
1 DEMOS.openShift.namespace=os_demo_system
2 DEMOS.openShift.username=os_user
3 DEMOS.openShift.password=os_pwd
4
```

Dadurch muss ein Testfall normalerweise keinen expliziten `oc login` ausführen.

7. Automatische Exit-Code-Prüfung

Standardmäßig gilt für EXEC :

```
1 expectedExitCode: 0
2 checkExitCode: true
3
```

Ein unerwarteter Non-Zero Exit-Code führt damit automatisch zu einem fehlgeschlagenen Testschritt.

Ein normaler Schritt:

```
1 - type: OC
2   op: EXEC
3   pod: "my-service"
4   command: "java -version"
5
```

entspricht funktional:

```
1 - type: OC
2   op: EXEC
3   pod: "my-service"
4   command: "java -version"
5   expectedExitCode: 0
6   checkExitCode: true
7
```

7.1 Erwarteter Non-Zero Exit-Code

Ein Non-Zero Exit-Code kann für einen Testfall das erwartete Ergebnis sein.

```
1 - type: OC
2   op: EXEC
3   pod: "my-service"
4   command: "sh -c 'exit 7'"
5   response: exit_result
6   expectedExitCode: 7
7
```

Der Schritt ist erfolgreich, wenn tatsächlich Exit-Code **7** geliefert wird.

7.2 Automatische Prüfung deaktivieren

Für Negativtests kann die automatische Prüfung deaktiviert werden:

```
1 - type: OC
2   op: EXEC
3   pod: "my-service"
4   command: "ls /file/that/does/not/exist"
5   response: failed_command
6   checkExitCode: false
7
```

Der Exit-Code wird trotzdem gespeichert.

Er kann anschließend explizit geprüft werden:

```
1 - type: OC
2   op: ASSERT
3   response: failed_command
4   action: EXITCODE
5   expected: 2
6
```

Damit stehen zwei Strategien zur Verfügung:

```
1 EXEC + expectedExitCode
2
```

für eine direkte Prüfung und

```
1 EXEC + checkExitCode:false
2 ASSERT EXITCODE
3
```

für eine spätere bzw. explizite Validierung.

8. Timeout

Mit `timeout` wird die maximale Laufzeit eines OC-Kommandos definiert.

Der Wert wird in Sekunden angegeben.

```
1 - type: OC
2   op: EXEC
3   pod: "my-service"
4   command: "sleep 10"
5   timeout: 1
6
```

Wird das Timeout überschritten, schlägt der Schritt fehl.

Ohne explizite Angabe gilt:

```
1 timeout = 30 Sekunden
2
```

9. outputFile

Der Output eines `OC EXEC` kann zusätzlich lokal gespeichert werden:

```
1 - type: OC
2   op: EXEC
3   pod: "my-service"
4   command: "java -version"
5   response: java_version
6   outputFile: "output/java-version.txt"
7
```

Wichtig ist die Unterscheidung zwischen lokalem und remote Dateisystem:

```

1 | OC command      -> läuft im Pod
2 | command-Pfade   -> befinden sich im Pod
3 |
4 | OC outputFile    -> befindet sich lokal auf dem AGATE-System
5 |

```

`outputFile` kopiert keine Datei aus dem Pod.

Es speichert ausschließlich den Output des ausgeführten Kommandos lokal.

Für einen echten Datei-Transfer werden `GET` und `PUT` verwendet.

Relative Pfade

Ein relativer Pfad wie:

```

1 | outputFile: "output/oc-output.txt"
2 |

```

wird relativ zum lokalen Arbeitsverzeichnis des AGATE-Prozesses aufgelöst.

Er bezieht sich **nicht** auf das Arbeitsverzeichnis des Pods.

10. PUT – Datei in einen Pod kopieren

`PUT` kopiert eine lokale Datei vom AGATE-System in einen Pod.

```

1 | - type: OC
2 |   op: PUT
3 |   pod: "my-service"
4 |   from: "C:\\tmp\\test.txt"
5 |   to: "/tmp/test.txt"
6 |   response: put_result
7 |

```

Dabei gilt:

```

1 | from -> lokal
2 | to   -> remote im Pod
3 |

```

Intern erfolgt der Transfer über `oc cp`.

10.1 PUT Parameter

Parameter	Beschreibung	Default
<code>pod</code>	Ziel-Pod	–
<code>namespace</code>	Ziel-Namespace	konfigurierte Umgebung
<code>from</code>	Lokaler Quellpfad	–

<code>to</code>	Zielpfad im Pod	–
<code>response</code>	Name des Transfer-Ergebnisses	–
<code>expectedExitCode</code>	Erwarteter Exit-Code	<code>0</code>
<code>checkExitCode</code>	Exit-Code automatisch prüfen	<code>true</code>
<code>timeout</code>	Timeout in Sekunden	<code>30</code>
<code>condition</code>	Optionale Bedingung	–

11. GET – Datei aus einem Pod kopieren

`GET` kopiert eine Datei aus einem Pod auf das lokale AGATE-System.

```

1 - type: OC
2   op: GET
3   pod: "my-service"
4   from: "/tmp/test.txt"
5   to: "C:\\tmp\\test.txt"
6   response: get_result
7
```

Dabei gilt:

```

1 from -> remote im Pod
2 to   -> lokal
3
```

Auch `GET` verwendet intern `oc cp`.

11.1 GET Parameter

Parameter	Beschreibung	Default
<code>pod</code>	Quell-Pod	–
<code>namespace</code>	Namespace	konfigurierte Umgebung
<code>from</code>	Pfad innerhalb des Pods	–
<code>to</code>	Lokaler Zielpfad	–

<code>response</code>	Name des Transfer-Ergebnisses	–
<code>expectedExitCode</code>	Erwarteter Exit-Code	<code>0</code>
<code>checkExitCode</code>	Exit-Code automatisch prüfen	<code>true</code>
<code>timeout</code>	Timeout in Sekunden	<code>30</code>
<code>condition</code>	Optionale Bedingung	–

12. Exit-Code-Prüfung bei PUT und GET

`PUT` und `GET` verwenden dieselbe Exit-Code-Logik wie `EXEC`.

Standard:

```
1 expectedExitCode: 0
2 checkExitCode: true
3
```

Damit kann ein fehlgeschlagener `oc cp`-Transfer nicht als erfolgreicher AGATE-Schritt weiterlaufen.

Beispiel:

```
1 - type: OC
2   op: PUT
3   pod: "my-service"
4   from: "C:\\tmp\\test.txt"
5   to: "/tmp/test.txt"
6   response: put_result
7
8 - type: OC
9   op: ASSERT
10  response: put_result
11  action: EXITCODE
12  expected: 0
13
```

Die zusätzliche `ASSERT`-Prüfung ist möglich, aber für einen normalen Happy Path nicht zwingend erforderlich, da der Exit-Code bereits beim `PUT` automatisch geprüft wird.

13. Windows-Pfade bei PUT und GET

Lokale Windows-Pfade können in der DSL normal angegeben werden:

```
1 from: "C:\\tmp\\test.txt"
2
```

bzw.:

```
1 to: "C:\\tmp\\test.txt"
2
```

Die OC Engine behandelt die für `oc cp` notwendige Windows-Pfadauflösung intern.

Der Testfall muss deshalb keine spezielle `oc cp` -Syntax kennen.

Remote-Pfade bleiben normale Linux-Pfade:

```
1 /tmp/test.txt
2 /cloud/pst/output/report.txt
3
```

14. ASSERT – Validierung

`ASSERT` validiert den Exit-Code oder Output eines gespeicherten OC-Ergebnisses.

Action	Beschreibung	Parameter
<code>EXITCODE</code>	Prüft den Exit-Code	<code>expected</code>
<code>CONTAINS</code>	Prüft, ob Text enthalten ist	<code>value</code>
<code>NOT_CONTAINS</code>	Prüft, ob Text nicht enthalten ist	<code>value</code>
<code>EQUALS</code>	Prüft auf exakte Übereinstimmung nach <code>trim()</code>	<code>value</code>
<code>NOT_EQUALS</code>	Prüft auf Abweichung nach <code>trim()</code>	<code>value</code>
<code>COUNT</code>	Prüft die Anzahl eines Suchbegriffs	<code>value</code> , <code>expected</code>

EXITCODE

```
1 - type: OC
2   op: ASSERT
3   response: java_out
4   action: EXITCODE
5   expected: 0
6
```

CONTAINS

```

1 - type: OC
2   op: ASSERT
3   response: java_out
4   action: CONTAINS
5   value: "openjdk"
6

```

NOT_CONTAINS

```

1 - type: OC
2   op: ASSERT
3   response: java_out
4   action: NOT_CONTAINS
5   value: "ERROR"
6

```

COUNT

```

1 - type: OC
2   op: ASSERT
3   response: java_out
4   action: COUNT
5   value: "OpenJDK"
6   expected: 2
7

```

15. BUFFER – Datenextraktion

Mit **BUFFER** können Informationen aus einem gespeicherten OC-Output extrahiert werden.

```

1 - type: OC
2   op: BUFFER
3   response: java_out
4   action: TEXT
5   name: java_version
6

```

Das Ergebnis steht anschließend über:

```

1 {B[java_version]}
2

```

zur Verfügung.

Wichtig: **name** sollte einen anderen Bezeichner als **response** verwenden, damit das ursprüngliche Kommandoergebnis einschließlich Exit-Code erhalten bleibt.

15.1 BUFFER Actions

Action	Beschreibung	Parameter
TEXT	Speichert den kompletten getrimmten	–

	Output	
FILTER	Speichert Zeilen, die einen Suchbegriff enthalten	value
LINE	Extrahiert eine Zeile; 0 = erste Zeile	value optional, Default 0
LAST_LINE	Extrahiert relativ vom Ende; 0 = letzte Zeile	value optional, Default 0
COUNT	Zählt das Vorkommen eines Suchbegriffs	value

FILTER

```

1 - type: OC
2   op: BUFFER
3   response: java_out
4   action: FILTER
5   value: "OpenJDK"
6   name: openjdk_lines
7

```

LINE

```

1 - type: OC
2   op: BUFFER
3   response: java_out
4   action: LINE
5   value: 0
6   name: first_line
7

```

LAST_LINE

```

1 - type: OC
2   op: BUFFER
3   response: java_out
4   action: LAST_LINE
5   value: 0
6   name: last_line
7

```

COUNT

```

1 - type: OC
2   op: BUFFER
3   response: java_out
4   action: COUNT
5   value: "OpenJDK"
6   name: openjdk_count
7

```

16. Vollständiges Beispiel

```
1  testCases:
2
3    - id: TC_OC_FULL_DEMO
4      description: "OC Engine - Full Happy Path"
5      stage: "*"
6      priority: HIGH
7
8      variables:
9        service: "my-service"
10       filename: "test.txt"
11
12     steps:
13
14       # EXEC
15       - type: OC
16         op: EXEC
17         pod: "{B[service]}"
18         command: "java -version"
19         response: java_out
20
21       # Output validieren
22       - type: OC
23         op: ASSERT
24         response: java_out
25         action: CONTAINS
26         value: "openjdk"
27
28       # Daten extrahieren
29       - type: OC
30         op: BUFFER
31         response: java_out
32         action: TEXT
33         name: java_version
34
35       # Erwarteter Non-Zero Exit-Code
36       - type: OC
37         op: EXEC
38         pod: "{B[service]}"
39         command: "sh -c 'exit 7'"
40         response: exit_7
41         expectedExitCode: 7
42
43       # Automatische Prüfung deaktivieren
44       - type: OC
45         op: EXEC
46         pod: "{B[service]}"
47         command: "sh -c 'exit 5'"
48         response: exit_5
49         checkExitCode: false
50
51       - type: OC
52         op: ASSERT
53         response: exit_5
54         action: EXITCODE
55         expected: 5
56
57       # Explizites Timeout
58       - type: OC
59         op: EXEC
60         pod: "{B[service]}"
61         command: "echo timeout test"
62         response: timeout_test
63         timeout: 5
64
```

```

65      # Output lokal speichern
66      - type: OC
67        op: EXEC
68        pod: "{B[service]}"
69        command: "echo AGATE OC outputFile test"
70        response: output_file_test
71        outputFile: "output/oc-output.txt"
72
73      # Lokale Datei erzeugen
74      - type: CMD
75        op: EXEC
76        command: 'echo foo > "C:\tmp\{B[filename]}"'
77
78      # Remote-Datei entfernen, falls vorhanden
79      - type: OC
80        op: EXEC
81        pod: "{B[service]}"
82        command: "rm -f /tmp/{B[filename]}"
83
84      # PUT
85      - type: OC
86        op: PUT
87        pod: "{B[service]}"
88        from: "C:\\tmp\\{B[filename]}"
89        to: "/tmp/{B[filename]}"
90        response: put_result
91
92      # Remote-Inhalt prüfen
93      - type: OC
94        op: EXEC
95        pod: "{B[service]}"
96        command: "cat /tmp/{B[filename]}"
97        response: remote_content
98
99      - type: OC
100        op: ASSERT
101        response: remote_content
102        action: CONTAINS
103        value: "foo"
104
105      # GET
106      - type: OC
107        op: GET
108        pod: "{B[service]}"
109        from: "/tmp/{B[filename]}"
110        to: "C:\\tmp\\{B[filename]}"
111        response: get_result
112

```

17. Troubleshooting & Best Practices

Pod nicht gefunden

Wenn kein passender Pod gefunden wird, prüfen:

- Ist der richtige Namespace aktiv?
- Ist der Service-/Pod-Präfix korrekt?
- Existiert ein laufender Pod?
- Ist der Pod im Status `Running` ?

Unerwarteter Exit-Code

Da `checkExitCode` standardmäßig `true` ist, führt ein unerwarteter Non-Zero Exit-Code automatisch zum Fehler.

Bei erwarteten Fehlerfällen:

```
1 expectedExitCode: 2
2
```

oder:

```
1 checkExitCode: false
2
```

verwenden.

Timeout

Lang laufende Kommandos können ein explizites Timeout erhalten:

```
1 timeout: 120
2
```

Das globale Standardverhalten sollte nur dann überschrieben werden, wenn das Kommando tatsächlich länger laufen darf.

PUT/GET schlägt fehl

Prüfen:

- lokalen Pfad,
- Remote-Pfad,
- Namespace,
- Pod,
- Schreib-/Leseberechtigungen,
- vorhandene Zielverzeichnisse.

Der Exit-Code von `oc cp` wird standardmäßig automatisch geprüft.

outputFile und GET nicht verwechseln

```
1 outputFile
2   = speichert stdout/stderr eines OC EXEC lokal
3
4 GET
5   = kopiert eine echte Datei aus dem Pod auf das lokale System
6
```

18. Kurzreferenz

```
1 oc
```

```
2  | EXEC
3  |   | pod
4  |   | namespace
5  |   | command
6  |   | response
7  |   | expectedExitCode [default: 0]
8  |   | checkExitCode   [default: true]
9  |   | timeout          [default: 30s]
10 |   | outputFile
11 |
12 | PUT
13 |   | pod
14 |   | namespace
15 |   | from [local]
16 |   | to   [remote]
17 |   | response
18 |   | expectedExitCode [default: 0]
19 |   | checkExitCode   [default: true]
20 |   | timeout          [default: 30s]
21 |
22 | GET
23 |   | pod
24 |   | namespace
25 |   | from [remote]
26 |   | to   [local]
27 |   | response
28 |   | expectedExitCode [default: 0]
29 |   | checkExitCode   [default: true]
30 |   | timeout          [default: 30s]
31 |
32 | BUFFER
33 |   | TEXT
34 |   | FILTER
35 |   | LINE
36 |   | LAST_LINE
37 |   | COUNT
38 |
39 | ASSERT
40 |   | EXITCODE
41 |   | CONTAINS
42 |   | NOT_CONTAINS
43 |   | EQUALS
44 |   | NOT_EQUALS
45 |   | COUNT
46 |
```